Basics

Part 1: Basic Concepts

1. What are the three primary criteria we seek when designing an algorithm?
2. List the five basic data structure types and their average time and space complexities.
3. What is the specific objective of the **Interval Scheduling Problem**?
4. List three different types of greedy strategies that could be used for interval scheduling.
5. What is the fundamental difference between an **algorithm** and a **heuristic**?

Part 2: Application and Analysis

1. Describe the **Earliest Finish Time** greedy algorithm steps.
2. Why is the **Nearest Neighbor** approach for the Traveling Salesman Problem considered a heuristic rather than a correct algorithm?
3. Explain the concept of a **Loop Invariant** and its relation to mathematical induction.
4. What does it mean for a problem like the Traveling Salesman Problem (TSP) to be **NP-hard**?
5. How does **modularity** contribute to an algorithm's ease of implementation?

Part 3: Proof and Complexity

1. **Proof by Contradiction:** Summarize the argument used to prove that the greedy algorithm for interval scheduling cannot select fewer jobs than an optimal schedule.
2. **Counterexample Challenge:** Draw or describe a set of intervals where the **Shortest Interval** greedy strategy fails to provide a maximum-size subset of compatible jobs.
3. **Exhaustive Search Complexity:** Why is exhaustive search (trying n! permutations) impractical for a Robot Tour Optimization with 20 points? Provide the approximate number of permutations involved.
4. **Algorithm Correctness:** The slides state that "Failure to find a counterexample... does not mean the algorithm is correct." Explain why mathematical induction is preferred over trial-and-error for proving correctness.
5. **Complexity Trade-offs:** Compare **Quicksort** and **Mergesort** based on their worst-case time complexity and space complexity.

Advanced Algorithms & Data Structures

Programming Assignment: Interval Scheduling

Empirical Runtime and Optimality Study

1 Objectives

By completing this assignment, students will:

- Implement and compare multiple **greedy algorithms** for interval scheduling.
- Implement an **exhaustive (exponential-time) algorithm** to compute the optimal solution for small instances.
- Design **controlled synthetic datasets** and understand how input distributions affect algorithm behavior.
- Empirically measure **running time growth** and compare it with theoretical **Big-O complexity**.
- Analyze **solution quality versus optimal** for different greedy criteria.
- Practice rigorous **experimental methodology** and result interpretation.

2 Problem Definition

You are given a set of n intervals:

$$I = \{(s_i, f_i)\}_{i=1}^{n}$$

where s_i is the start time and f_i is the finish time, with $s_i < f_i$.

Two intervals are **compatible** if they do not overlap:

$$f_i \leq s_j \quad \text{or} \quad f_j \leq s_i$$

Goal: Select a maximum-size subset of pairwise non-overlapping intervals.

3 Algorithms to Implement

3.1 Greedy Algorithms (Polynomial Time)

Implement the following greedy strategies. Each algorithm must:

1. Sort intervals by the specified criterion
2. Scan in sorted order and select the next compatible interval

Required greedy criteria

- **Earliest Finish Time (EFT):** sort by increasing f_i
- **Earliest Start Time (EST):** sort by increasing s_i
- **Shortest Duration (SD):** sort by increasing $(f_i - s_i)$

Each greedy algorithm should return:

- the number of selected intervals
- (optional) the selected set of intervals

Theoretical runtime:

$$O(n \log n)$$

3.2 Exhaustive Algorithm (Exponential Time)

Implement an algorithm that computes the **true optimal solution**. This algorithm is required only for small values of n.

Choose **one** of the following approaches:

Subset Selection

- Precompute interval compatibility
- Enumerate all subsets and keep the largest feasible one

Worst-case time:

$$O(n2^n)$$

The exhaustive algorithm will be used as an **optimality oracle** for validating greedy solutions.

4 Dataset Generation

4.1 Choosing the Time Horizon T

Let:

- n be the number of intervals
- D be the maximum (or expected) interval duration

To ensure meaningful and comparable experiments, the time horizon T must scale with n.

Default rule:

$$\boxed{T = \alpha \cdot n \cdot D}$$

where $\alpha > 0$ controls the **overlap density**.

This choice keeps the expected overlap behavior approximately stable as n increases.

4.2 Overlap Regimes

Students must test **all three** overlap regimes.

α	Regime	Description
$\alpha \approx 0.1$	High overlap	Dense conflicts
$\alpha \approx 1$	Medium overlap	Balanced conflicts
$\alpha \approx 5$	Low overlap	Sparse conflicts

4.3 Required Dataset Types

Uniform Random Dataset

- Choose D (e.g., 10 or 100)
- Set $T = \alpha \cdot n \cdot D$
- Generate:
$$s_i \sim \text{Uniform}[0, T), \quad d_i \sim \text{Uniform}[1, D], \quad f_i = s_i + d_i$$
- for high-overlap dataset, choose $\alpha = 0.1$
- for medium-overlap dataset, choose $\alpha = 1$
- for low-overlap dataset, choose $\alpha = 5$

5 Experimental Protocol

5.1 Input Sizes

Greedy algorithms:
$$n \in \{2^{10}, 2^{11}, \ldots, 2^{20}\}$$

Exhaustive algorithm:
$$n \in \{5, 10, 15, \ldots, n_{\max}\}$$

where $n_{\max}$ is the largest size that completes in reasonable time.

5.2 Trials

- Minimum of 10 trials per configuration
- Report mean and standard deviation

5.3 Timing Rules

- Exclude data generation time
- Use high-resolution timers
- Perform at least one warm-up run

6 Big-O Validation Methodology

6.1 Greedy Algorithms

Expected complexity:

$$O(n \log n)$$

Required plots:

- Runtime $t(n)$ vs. n (log–log scale)
- Normalized runtime:

$$\frac{t(n)}{n \log_2 n}$$

6.2 Exhaustive Algorithm

Expected complexity:

$$O(n2^n)$$

Required plots:

- Runtime $t(n)$ vs. n
- Normalized runtime:

$$\frac{t(n)}{n2^n}$$

Students must explain why pruning may reduce runtime for small n but does not change the worst-case complexity.

7 Expected Outcomes

7.1 Solution Quality

- **Earliest Finish Time** always matches the optimal solution.
- **Earliest Start Time** and **Shortest Duration**:
  - Often match the optimum in low-overlap datasets
  - May be suboptimal in high-overlap datasets

7.2 Runtime Behavior

- All greedy variants scale similarly.
- $\frac{t(n)}{n \log n}$ approaches a constant.
- Exhaustive runtime grows rapidly and exhibits exponential behavior.

7.3 Effect of T

- Smaller T leads to higher overlap and larger quality differences.
- Larger T leads to sparse conflicts and similar greedy behavior.
- Incorrect scaling of T leads to misleading conclusions.

8 Deliverables

- Source code:
  - dataset generator
  - greedy algorithms
  - exhaustive solver
  - benchmarking
- Report:
  - algorithm descriptions
  - complexity analysis
  - justification of T
  - plots and discussion
- Artifacts:
  - Results
  - plots (PDF or PNG)

9 Grading Rubric

Component	Weight
Correctness & feasibility	30%
Experimental rigor	20%
Big-O validation	25%
Comparative analysis	15%
Code quality	10%